Job Scout: a guarded, private job search

An automated job search and application tracker that runs every morning on my own mailbox and career record, designed from the start so that it can never act for me.

Rory McClelland · mcclelland.info

Version 0.12 · October 2026

30 min
of my review each morning; I make every decision

3
AI agents around a core of ordinary, testable code

21 of 22
adverts read correctly by a local model with no tools

10 of 10
hostile red-team inputs stopped in the first exercise

Job Scout runs every morning. It reads my job alerts and the replies to my applications, finds and reads the full adverts, scores each role against criteria I wrote, drafts a CV and covering letter for the best few, has a second AI agent check every claim in those drafts against my career record, and puts the results on a private board where I decide what to do. It never applies for a job, replies to an email or contacts anyone. I do those things myself.

Using AI to write a covering letter is not new. The interesting part is the design: how to make a system of AI agents safe enough to run every day on a private mailbox and a career record, and how to prove that it is. I designed the system and made every decision; Claude wrote the code and the agents' instructions to my specification.

01 The problem, and why it needs guardrails

A senior job search is a daily grind of alert emails, most of them irrelevant, and the relevant ones each need a tailored CV and letter. It is ideal work for automation. The data, however, is exactly the kind that should not be handed to automation casually:

- **A private mailbox**, full of other people's personal data as well as my own.
- **A career record** from which every claim made in my name is drawn; an overstated application cannot be withdrawn.
- **Text written by strangers**: adverts, alerts and replies, any of which may carry hidden instructions aimed at AI screening tools, or be an outright scam.
- **A reputation with employers**, which one message sent without my knowledge could damage.

So the frame was agreed before anything was built: the job (*"run a senior job search in 30 minutes a morning"*), two success measures (relevant roles found per day, and applications actually made), a human in the loop on every role and every draft, and a never list.

NEVER
apply for a role

NEVER
reply to or follow up any message

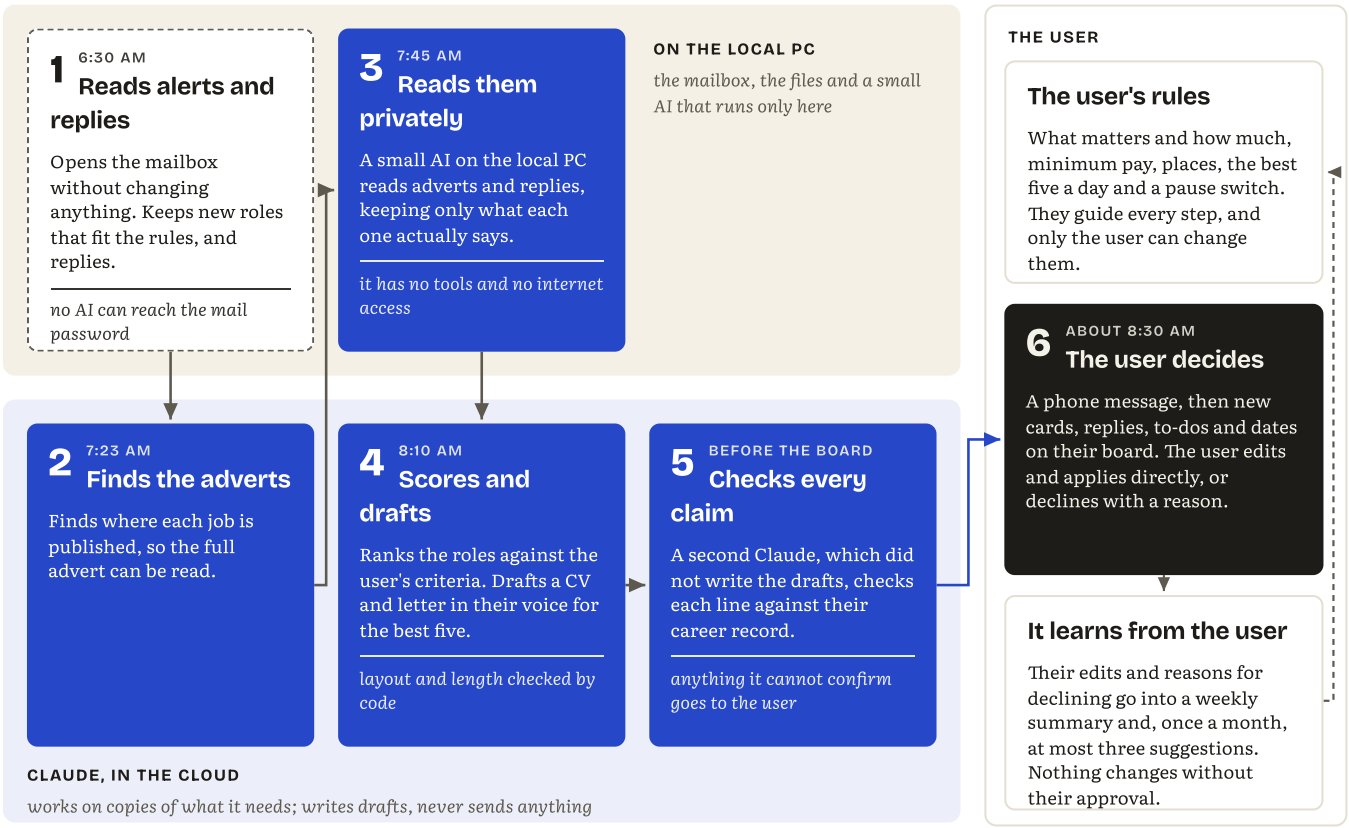
NEVER
edit my LinkedIn profile or a sent document

NEVER
share the board or change the rules

02

How a morning runs

The work is split between the local PC and Claude in the cloud, on a fixed schedule, and ends with me. Each blue step is AI; the dashed step is ordinary code.



Stays on the local PC

- the mailbox and the emails themselves
- the mail password
- replies: only a short summary leaves

What Claude works with

- the facts of each job, checked first
- the user's career notes and writing style
- the drafts it writes

Never happens

- applying, replying or accepting anything
- editing LinkedIn or a sent document
- changing the rules without the user's approval

The morning run. Mail is read on the local PC at 6:30; the cloud finds the adverts; a model on the local PC reads them privately; the cloud scores, drafts and independently checks; I review on the board at around 8:30. The three panels at the foot are the data boundaries and the never list.

03

Three design choices

Isolation, independence or on-demand use: otherwise, ordinary code.

1. An AI agent only where one is needed

Each extra agent adds cost, more hand-over points and more ways to fail. So a step becomes an agent only for one of three reasons: **isolation** (it reads text written by strangers, so it must hold nothing valuable and have no tools that could cause harm), **independence** (it judges another step's work, so it must be separate from that step), or **on-demand use**. Everything else is ordinary code: reading mail, filtering, removing duplicates, counting applicant-tracking-system keywords, matching replies to applications, and every check on what the AI produces.

2. Each kind of data has a boundary

The design starts from one question: which part of the system may see which data?

DATA	WHO CAN SEE IT	KEPT FOR
My mailbox	Only code on the local PC, which can read mail but cannot change or send anything.	Not copied, except as below
Copies of emails	Code on the local PC, in a private store outside every folder an AI can reach.	30 days
Adverts and replies, written by strangers	A model on the local PC with no tools and no internet. Only a checked summary leaves the PC.	90 days, unless I applied
My career record	The writer and the reviewer, in the cloud. The reviewer can read but not write.	My own files
My mail password	Nothing. It is encrypted for my Windows account, outside every folder an AI can reach.	Until changed

The local model was chosen for safety, not to save money. An advert may contain hidden instructions, but the model that reads it has no tools, so it cannot act on them; code then keeps only the facts that appear in the advert's own words. Board cards never hold a contact's email address or phone number, and strangers' emails are never sent to the cloud.

3. Several layers of protection

No single control is trusted on its own. There are five layers:

LAYER 1

Text is treated as data

Before any AI sees an alert, advert or reply, code removes lines that look like instructions and reports what it removed. The AI is told to treat what remains as material to assess, never as instructions.

LAYER 2

Code checks before anything is saved

Every score, draft, claim, reply match and board entry is checked by code before it is written. If a check fails, nothing is written and the problem is reported.

LAYER 3

The reviewer must show evidence

For every claim it accepts, the independent reviewer must quote my career record, and code confirms that the quote really exists. The review never rests on the reviewer's word alone.

LAYER 4

Changes to the board are earned

At first the system may only suggest changes. It may move a card by itself only after I have tested its reply reader on my own replies. Every move is logged and can be undone.

LAYER 5

Limits, an off switch and a ledger

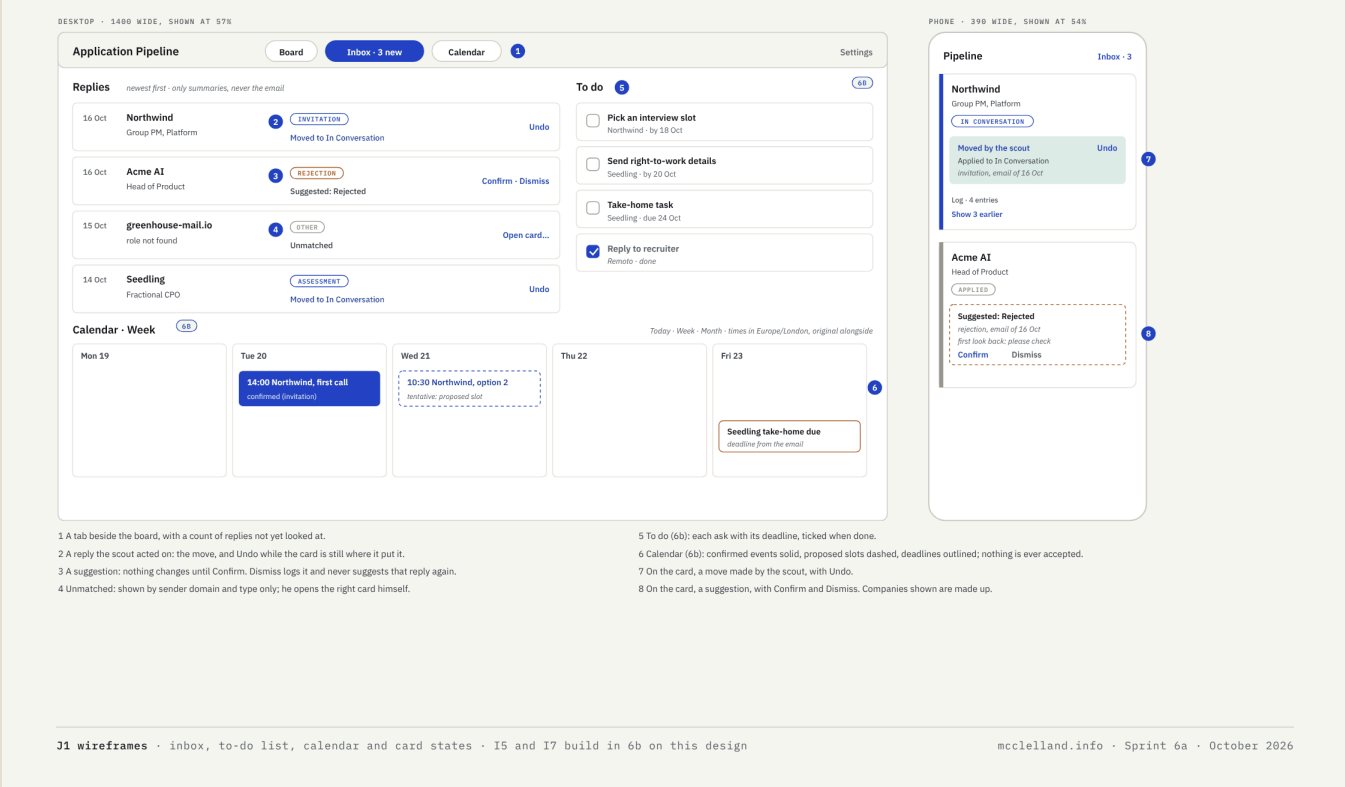
A pause switch on the board stops every part of the system, and daily caps limit the adverts read and drafts written (which also caps the running cost). A ledger records every step: which model ran, with which version of its instructions and which inputs, so any output can be traced back to how it was made.

04 The application tracker

The board, which I call the Application Pipeline, is where the system meets me. It is a private web page with its own database, visible only to a signed-in viewer, and it is the place I work

each morning.

- **A pipeline in four columns:** Draft, Applied, In Conversation and Rejected, with a card per role carrying a short paragraph on the company and the role, the next step, the date applied, notes and a stamped log. Scouted roles arrive in Draft, with their score, warning signs (an agency poster, an unnamed client, a repost, a very high applicant count) and a link to the prepared pack.
- **Replies become an inbox, a to-do list and a calendar.** Replies are read on the local PC, matched to cards by fixed rules in code, and shown as summaries, never the email itself. Interview slots proposed in a reply appear as tentative; nothing is ever accepted.
- **Suggestions until trust is earned.** A rejection appears as "Suggested: Rejected" with Confirm and Dismiss, so nothing changes until I say so. Warnings travel with the reply, for example "asks for money or bank details".
- **The rules are mine alone.** Every setting (weights, threshold, caps, pay floors, places) is editable on a Settings screen that checks each value on entry and logs every change. No AI can write to it.



The inbox wireframes, agreed before the screens were built: a move the scout made (with Undo), a suggestion waiting for me (Confirm or Dismiss), and an unmatched reply shown only by sender domain and type. Companies shown are invented.

Two smaller decisions say something about the approach. The weekly histogram of applications is a single series by choice, because colouring it by stage failed a colour-blind separation check. And a self-hosted public version was built and then dropped the same day: a static site would have been publicly reachable, and a board holding candid notes on each application should not be.

05 Testing: results are measured, not assumed

Every AI component has a bar set before it is measured, and the evaluations are re-run before any change to a model, the scoring criteria or an agent's instructions.

WHAT IS TESTED	BAR	RESULT
Local reader: requirements and pay copied correctly	9 in 10	21 of 22 adverts
Reviewer: planted problems found (a planned result written as an achievement, a barred claim, an instruction hidden in a keyword)	All	8 of 8 cases
Red team: hostile inputs stopped	All	10 of 10
Automated tests, using made-up and hostile emails and adverts	All pass	131 code tests, 113 tool tests
Reply reader agrees with my own grading	9 in 10, and no rejection read as an invitation	Not yet graded, so every board move it causes stays a suggestion

The red team

Every quarter, ten hostile, made-up inputs go through the real system: hidden instructions in alerts, email subject lines and adverts; a request to send data elsewhere; a fake recruiter; and a fake job offer asking for bank details. All ten failed. The exercise also found four genuine bugs, each logged and since fixed or scheduled. One was that an instruction placed in an email's subject line reached the cloud without being removed; another, that a warning on a reply ("asks for bank details") did not appear on the board.

An honest result

The first scoring evaluation, comparing the system's scores for 34 roles with my own apply-or-pass decisions, taught me more about the test than about the scorer: most adverts had not been saved, so the roles were re-scored from the short alert alone. The set is being rebuilt from runs that save every advert, and no baseline will be set until it is. I would rather publish that than a flattering number.

06 What only real use found

The first live mornings brought replies and to-dos onto the board, and with them the problems that only real use finds. The local PC's power-saving mode blocked the scheduled wake-up and switched off the network, so runs started late; it no longer sleeps on mains power. A graphics setting left the local model on the main processor at about a tenth of its speed, which only the reader's own speed warning revealed. Both fixes are in the runbook. AI systems often fail where they meet the real world, not in their reasoning, and they need the same operational care as any other service.

07 How it was delivered

Fourteen releases in four days of building, then live running and fixes. Every sprint ended in working software, was refined to agreed test cases before building started, and was checked against them at the close. Since Sprint 7, a separate agent that has not seen the work reviews the code before each delivery; on its first outing it found eleven issues, nine fixed before release and two accepted with reasons.

PHASE	WHAT IT DELIVERED
A working search	Read-only mail reading, parsers, filters with a reason for every drop, a scoring rubric, CV and letter packs built from my own documents, and Draft cards on the board. The first real run read 48 emails and 209 roles, and shortlisted five.
Contain the AI	A risk review of the working system before extending it: the mail password out of reach of every AI, alert text treated as data, every output checked by code, and the pause switch.
Read privately, check every claim	A spike to test whether a local model was good enough, a private email store, an audit trail, the tool-less local reader, and the independent reviewer.
Track replies	Replies found by sender address and signature (never display name), read on the PC, matched to cards; then the inbox, to-do list and calendar.
Measure, not assert	Evaluations, the red team, retention limits, a risk register organised by the NIST AI Risk Management Framework, and the independent code review.

08 The documentation set

Kept with the code and reviewed at every sprint close: a system card, an agent catalogue, a threat model, a risk register, a numbered decision log, a runbook, a requirements and configuration reference, architecture diagrams for general and technical readers, and a sprint-by-sprint delivery history.

Next: a demo mode running on a fictional candidate, so the system can be shown without exposing anyone's data; scoring or drafting a single role on request; and a "worth a look" band for roles that fall just below the line.

Built with: TypeScript on Node.js with no external libraries for the PC; Python standard library for the checking tools; the qwen2.5:14b model run locally with Ollama on the local PC's graphics card; Claude scheduled tasks for the cloud steps; and a private board built in Claude with its own database.

Rory McClelland · product leader, AI products and governance
hello@mcclelland.info · mcclelland.info

Companion case study:
How I build with AI: guardrails first